

SQUADLINK

Local Commerce & Last-Mile Delivery Coordination Platform

Initial market focus: Bauchi State, Nigeria

This document defines the Minimum Viable Product (MVP) that SquadLabs will build and demonstrate for Hackathon 2.0. It intentionally excludes long-term/future features that are documented separately in the SQUADLINK Master Business & System Design.

1. Document Information & Product Summary

Field	Detail
Product Name	SQUADLINK
Solution Type	Software – Responsive Web Application, with a dedicated mobile-optimized rider interface
Prepared By	SquadLabs
Document Version	1.0
Date	September 2026

Product Summary

SQUADLINK is a web-based local commerce and last-mile delivery coordination platform connecting customers with registered local businesses and independent delivery operators in Bauchi State. Customers browse a unified product catalogue, and before any order is finalized and paid for, SQUADLINK verifies that a single registered business can fulfill the complete order based on product availability, quantity, pricing and business status. Once a business is confirmed, SQUADLINK dispatches an independent rider — motorcycle or Keke NAPEP — to collect and deliver the order, using pickup-code and delivery-OTP verification for accountability.

SQUADLINK does not own the participating businesses and does not operate its own delivery fleet. It is a digital coordination layer connecting customers, local businesses and independent delivery operators.

Why This Document Exists

This PRD converts SQUADLINK's project idea into a clear, testable MVP plan for the hackathon committee. It defines the problem being solved, the intended users, what the MVP must do, how success will be judged, and the boundary of what SquadLabs intends to build during the competition window.

2. Problem Statement

Customers in Bauchi often want products from nearby local shops but cannot always travel to purchase them. Many local businesses have products and customers but lack a structured digital ordering and delivery channel. Independent bike and Keke NAPEP operators often have idle capacity that is not connected to delivery demand. Existing informal delivery arrangements offer limited visibility, verification, accountability and operational data, leaving customers uncertain about product availability, delivery status, delivery cost and order completion.

Problem Evidence / Assumptions to Validate

- Customers need a simple way to browse a local catalogue and place a trackable delivery order from a phone or computer.

- An order should only be confirmed once a single business is verified able to supply the complete order, to avoid part-filled or failed deliveries.
- Pickup and delivery verification (pickup code + OTP) can materially increase trust and accountability compared to informal arrangements.
- Independent riders (bike and Keke NAPEP) represent available capacity that can be coordinated rather than newly created.

Note: as with any hackathon PRD, these assumptions should be validated further through informal interviews, observation of existing local delivery arrangements, and pilot data once available.

3. Product Goal & Objectives

Product Goal

To provide a simple digital channel through which Bauchi customers can order from local businesses with confidence that the full order can be fulfilled, and through which independent riders can be coordinated to complete verified pickup and delivery.

Project Objectives

ID	Objective	How Completion Will Be Gauged
01	Enable a customer to browse the catalogue, build an order and complete a simulated checkout.	A test customer can select products, enter a delivery location and complete a sandbox payment.
02	Verify that a single business can fulfil the complete order before payment is finalized.	The Fulfillment Engine selects one eligible business or returns a clear no-business-found result.
03	Coordinate an available bike or Keke NAPEP rider to collect and deliver the order.	The Dispatch Engine assigns a suitable rider who can accept or decline the delivery.
04	Verify pickup and delivery using a pickup code and a delivery OTP.	Business verifies the pickup code; rider verifies the delivery OTP before the order completes.
05	Record business settlement and rider earnings in a ledger once delivery is verified.	Ledger entries appear as eligible/pending settlement after a completed order.
06	Give an administrator basic monitoring and exception-handling capability.	Administrator can view an order's full event history and resolve a basic exception case.

4. Target Users & Personas

User	Need	Typical Action	Success Condition
Customer	Convenient access to local products with delivery confidence	Browse catalogue, order, check out, track, confirm delivery	Can order in a few steps and trust the order will be fulfilled and delivered
Business	A structured digital demand channel	Maintain catalogue, receive fulfillment requests, verify pickup	Can manage catalogue and fulfil orders without manual coordination
Rider	Access to delivery income using existing bike/Keke capacity	Set availability, accept deliveries, verify pickup/delivery	Can find and complete deliveries and see recorded earnings

User	Need	Typical Action	Success Condition
Administrator	Operational oversight and exception control	Monitor orders, approve catalogue changes, reassign riders	Can inspect and resolve any order without direct database access

Primary Persona Example

Musa, 24, lives in Bauchi and often needs household items and drinks delivered without leaving his shop during work hours. He wants to search nearby businesses, confirm a product is actually in stock before he pays, and know exactly when his order has been picked up and is on its way, without having to call anyone to check.

5. Value Proposition & Significance

SQUADLINK is not simply a digital order form; it structures the whole local-delivery loop so that fulfillment is verified before payment, delivery is verified at both ends, and every participant has clear visibility into the process.

- For customers: confidence that an order will actually be fulfilled, plus visibility into order progress.
- For businesses: a digital demand channel and a catalogue that only accepts orders it can genuinely fulfil.
- For riders: access to organized delivery income using vehicles they already operate.
- For the SquadLabs team: a demonstrable, end-to-end coordination system that proves the underlying business mechanism.

6. Proposed Solution & User Experience

The MVP is a responsive web application for customers, businesses and administrators, with a mobile-optimized interface for riders. The core experience follows one straight-line path: product discovery → complete-order business verification → payment → business fulfillment → rider assignment → pickup → delivery → OTP confirmation.

Customers do not need to know which business will fulfil their order in advance — they select products and a delivery location, and the Fulfillment Engine identifies a single capable business before checkout is finalized. If no nearby business can fulfil the complete order, the search radius expands progressively before a clear failure is returned.

SQUADLINK [Menu] [Cart 2]

[Search products...] [Search]

Delivery location: [Set location]

Categories:

Household

Drinks

Food

Electronics

[product image]

Product name
Price [+ Add]

[product image]

Product name
Price [+ Add]

[product image]

Product name
Price [+ Add]

[product image]

Product name
Price [+ Add]

Fulfilled by: [Nearby Business Name]

Order summary

Subtotal NGN X,XXX
Delivery charge NGN X,XXX
Total NGN X,XXX

[Proceed to checkout]

[Home] [Orders] [Track] [Profile]

Figure 1. Low-fidelity wireframe — customer catalogue & checkout screen.

NOTE: A Week 1 wireframe does not need to be visually polished; it should demonstrate layout, hierarchy and intended interaction. Additional screens (business catalogue view, rider delivery-offer screen) follow the same layout logic and will be produced as the design work progresses.

7. Functional Requirements

ID	Requirement	Priority	Primary User	Acceptance Summary
FR-01	Register/log in and select a language	Must	Customer/Business/Rider	Account created; language preference stored
FR-02	Browse/search the unified catalogue and select products and quantities	Must	Customer	Products with image, price and business shown; items addable to order
FR-03	Enter a delivery location and view a calculated order total	Must	Customer	Total reflects product, delivery and applicable charge fields
FR-04	Verify a single business can fulfil the complete order before checkout	Must	Platform	Order proceeds only once one business is confirmed for all items
FR-05	Complete a simulated/sandbox payment	Must	Customer	Order shows PAYMENT_VERIFIED on success; PAYMENT_FAILED handled otherwise
FR-06	Maintain a business catalogue with price, availability and last-updated data	Must	Business	Only items marked available are considered fulfillable
FR-07	Select an alternative business if the first cannot fulfil the complete order	Must	Platform	Fulfillment Engine retries/expands radius before returning failure
FR-08	Match and dispatch an available bike or Keke NAPEP rider	Must	Platform	Suitable rider notified; can accept or decline
FR-09	Reassign to another rider on decline/no response	Must	Platform	A new suitable rider is offered the delivery
FR-10	Generate and verify a pickup code at the business	Must	Business/Rider	Order only proceeds to in-transit after business verifies the code
FR-11	Verify delivery via OTP entered by the rider	Must	Customer/Rider	Order completes only after correct OTP verification
FR-12	Record business settlement and rider earnings in a ledger	Must	Platform	Ledger entries appear as eligible/pending after completion
FR-13	Provide administrator monitoring, approvals and exception handling	Must	Administrator	Admin can view, approve catalogue changes and

ID	Requirement	Priority	Primary User	Acceptance Summary
				reassign/resolve exceptions
FR-14	Notify participants in-app/web of key order events	Should	All	Relevant party notified on major status changes
FR-15	SMS notification for key delivery events	Could	Rider/Customer	Sent only if time/resources permit

8. Non-Functional Requirements

Area	Requirement	Target / Constraint	Verification
Usability	Core customer ordering flow should be simple on mobile web	Order placement achievable in a small number of primary steps	Task walkthrough
Performance	Common pages should respond promptly under demo load	Most requests complete quickly in the test environment	Basic timing tests
Security	Backend enforces authentication and role-based authorization	No privileged endpoint accessible without valid role	Access-control tests
Data validation	All user-supplied data validated server-side	Invalid/incomplete submissions are rejected with clear errors	Negative tests
Reliability	Order lifecycle uses explicit states, not ad-hoc flags	No silent state changes; exceptions recorded as controlled transitions	State-machine test checklist
Privacy	Collect and expose only data required per role	Customer payment data hidden from riders; rider identity hidden from customers	PRD/data review

9. MVP Scope & Prioritization

The MVP is the smallest credible version of SQUADLINK that proves the full coordination loop: catalogue-based ordering → complete-order business verification → payment → fulfillment → dispatch → verified pickup → verified delivery → settlement recording.

Priority	Included	Hackathon Decision
Must	Customer registration/login, catalogue browsing, cart, delivery location, sandbox checkout, single-business fulfillment verification, alternate-business fallback, business catalogue management, rider registration/availability, bike/Keke dispatch, pickup code, delivery OTP, order-status tracking, admin monitoring/exception handling, RBAC, audit logging, input validation.	Build and test first
Should	In-app/web notifications on major status changes; provision for English/Hausa localization; basic 3D-generated visual assets for product/demo presentation.	Build after Must features are stable
Could	SMS notifications where time/resources permit; interactive 3D (GLB/GLTF) elements if frontend capacity allows.	Only if time remains
Out of Scope	Live GPS tracking, production USSD/IVR, Hausa voice automation, automated real-money payouts, multi-business/split orders, AI-based dispatch, dynamic pricing, advanced analytics, multi-city operations.	Documented for future work

10. System Architecture & Technical Design

Proposed Technology Layers

Layer	MVP Requirement
Presentation	Responsive web app for customer/business/admin; dedicated mobile interface for riders.
Backend/API	Authentication, RBAC, users, businesses, catalogue, orders, fulfillment, dispatch, notifications and ledger APIs.
Database	Users, roles, businesses, products, catalogue items, riders, vehicles, orders, order items, delivery events, payments, ledger and audit logs.
Core logic	Catalogue Engine, Fulfillment Engine and Dispatch Engine implemented as clear, separate backend modules.
Notifications	In-app/web notifications as baseline; SMS only if time/resources permit.
Security	Authentication, server-side authorization, validation, protected secrets, audit logging, minimum-necessary data exposure.
Deployment	Reproducible demo environment with setup documentation for judges/test users.

Core Data Entities

- User — id, role, name, contact, language, authentication data.
- Business — id, owner, type, location, status, reliability.
- Product — id, name, category, image.
- CatalogueItem — business, product, price, availability, last_updated, approval status.
- Rider — user, vehicle type, capacity class, availability, reliability.
- Order — customer, business, delivery location, status, totals, timestamps.
- OrderItem — order, product, quantity, unit price, subtotal.
- Delivery — order, rider, pickup code, delivery OTP, delivery events.
- Payment — order, amount, status, reference.
- LedgerEntry — order, beneficiary, amount, type, status.
- AuditLog — actor, action, entity, timestamp, reason/result.

Key Technical Decisions

- One order = one business for the MVP; multi-business/split orders are excluded.
- Vehicle type is a simple rider attribute (bike/Keke NAPEP) rather than a separate logistics system.
- Order status changes use an explicit state machine rather than boolean flags, so exceptions are controlled transitions, not silent overrides.
- Payments are sandboxed/simulated for the hackathon; production payment and payout require separate provider and legal review.

11. Order Lifecycle

ORDER_CREATED → PAYMENT_VERIFIED → BUSINESS_SELECTED → RIDER_SEARCHING → RIDER_ASSIGNED → RIDER_ACCEPTED → AT_PICKUP → PICKUP_VERIFIED → IN_TRANSIT → DELIVERY_OTP_VERIFIED → COMPLETED → SETTLEMENT_PENDING

Controlled Exception Events

- PAYMENT_FAILED
- NO_BUSINESS_FOUND

- ITEM_UNAVAILABLE
- RIDER_DECLINED
- RIDER_NO_RESPONSE
- DELIVERY_FAILED
- ADMIN_INTERVENTION

Exceptions are recorded and routed through controlled transitions; they never silently bypass the order lifecycle.

12. Acceptance Criteria & Success Metrics

Objective / Area	MVP Acceptance Criterion	Pass Condition
Ordering & checkout	Customer can browse, order and complete a sandbox payment	Order reaches PAYMENT_VERIFIED with correct totals
Fulfillment	A single eligible business is verified before payment is finalized	Fulfillment Engine returns one business, or a clear no-business-found result
Dispatch	A suitable available bike or Keke rider is matched and can accept	Delivery reaches RIDER_ACCEPTED with a suitable rider
Pickup verification	Business verifies the pickup code before release	Order reaches PICKUP_VERIFIED only after code verification
Delivery verification	Rider verifies the delivery OTP	Order reaches COMPLETED only after correct OTP entry
Settlement	Business/rider settlement amounts recorded in the ledger	Ledger shows eligible/pending entries after completion
Exception handling	At least one controlled exception is demonstrated end-to-end	Rider decline or item-unavailable case resolves without manual code intervention

13. Risks, Constraints & Assumptions

Risk / Constraint	Impact	Likelihood	Mitigation
Limited hackathon build time (3–4 weeks)	High	High	Freeze Must-have scope early; defer Should/Could features
No live payment/payout provider	Medium	High (by design)	Use sandbox/simulated payment; document production path separately
Rider or business no-show during demo	Medium	Medium	Rehearse with seeded demo data and a scripted fallback path
Catalogue data becoming stale/incorrect	Medium	Medium	Track last-updated field; flag mismatches for admin review
Unauthorized access to admin/business functions	High	Low–Medium	RBAC enforced server-side; access-control tests before demo

Assumptions

- The MVP is a working prototype for the hackathon, not a production logistics deployment.
- Test users have internet access and a modern browser.
- Businesses and riders in the demo are represented by prepared test/demo accounts.
- Naira delivery-fee values and radius parameters are configurable placeholders, not final pricing.

14. Development Plan & Milestones

Period	Target
Days 1–2	Freeze requirements, architecture, data model, user journeys, GitHub setup and team responsibilities.
Days 3–5	Project skeletons, authentication, database/API foundation, navigation and initial catalogue.
Days 6–10	Customer ordering, business catalogue/order views, rider profile/availability and basic admin.
Days 11–14	Working happy path: customer → business → rider → pickup → delivery.
Week 3	Fulfillment/dispatch logic, radius search, pickup code, delivery OTP, ledger, notifications and localization scaffolding.
Week 4 (if available)	Security testing, exception cases, UI polish, 3D asset integration, deployment, documentation and demo rehearsal.

15. Open Questions / Decisions Pending

Question	Current Decision / Direction
Should customers be required to create accounts?	Yes for MVP, to support order tracking, delivery confirmation and repeat use.
How many businesses/riders will be seeded for the demo?	To be finalized during Days 1–2 planning; small, controlled demo dataset.
What is the maximum search-radius boundary?	To be configured as a parameter; exact value to be set during implementation.
Will SMS notifications be attempted?	Only if time/resources permit after core Must features are stable.

16. Change Management During the Hackathon

This PRD is a baseline for SQUADLINK's MVP, not a fixed requirement that every implementation detail remains unchanged. Where the team discovers technical limitations, integration challenges or better approaches, changes may be made with reasonable justification. For any meaningful change, the team will document what changed, why, what evidence led to it, and its impact on objectives, requirements or acceptance criteria — consistent with the change-management approach outlined by the hackathon organizers.